

3 Ways Java Functions Can Be Used in .NET

This demo shows three practical patterns for making Java logic available to a .NET app: JNI, IKVM, and gRPC. Each option has a different tradeoff in speed, compatibility, and ease of integration.

JNI

Java Native Interface

.NET directly loads the Java VM and calls Java classes from native code. This is the closest to direct runtime interop.

IKVM

Java Interop via IKVM

A Java jar is converted into .NET-friendly assemblies, letting .NET call Java classes as if they were native .NET classes.

gRPC

Remote Service Call

Java exposes a service over HTTP/2, and the .NET application sends a request and gets a result back over the network.

Comparison Table

Approach	Advantages	Disadvantages	Performance	Ease of Doing
JNI	Very low latency, direct access to Java runtime, good for performance-sensitive local integration.	Harder to maintain, requires native setup, JVM lifecycle management, and careful memory handling.	Very high for in-process calls	Hard — more native code and platform complexity
IKVM	Nice .NET developer experience, easier invocation from managed code, good for bridging existing Java libraries.	Requires Java jars and compatible runtime dependencies, can have compatibility issues with some Java APIs.	High , usually near native managed call speed	Medium — easier than JNI but still needs setup
gRPC	Strong separation of concerns, cross-language and cross-platform, easy to scale and deploy independently.	Network overhead, service ownership, and extra operational complexity for deployment and diagnostics.	Medium — best for remote services rather than tight local calls	Easy to model and consume in modern .NET apps

Example Modules in This Workspace

javaAPIs

This is the shared Java library that contains the Java implementation being accessed from .NET. The key logic is a method named **customAdd(int a, int b)**, which simply adds two numbers and returns the result.

dotNetJavaWithJNI

This module loads the JVM using JNI and invokes the Java class directly from .NET. It demonstrates the lowest-level way of calling Java functions by creating a JVM instance and then calling a Java method from managed code.

dotNetJavaWithkvm

This module shows the IKVM approach. It uses the Java API as a .NET-compatible library so the code can call Java classes like normal .NET objects instead of manually managing JNI function pointers and native VM setup.

javaAPIService

This is the Java service project. It exposes a gRPC endpoint that adds two integer values and returns the result. It is the remote service used by the .NET client to demonstrate inter-process interoperability.

dotnetJavaWithgRPC

This is the .NET client that calls the Java gRPC service at localhost on port 50051. It creates a client channel and sends an AddRequest, receiving an AddResponse with the computed sum.

Which One to Choose?

- **Choose JNI** when you need the fastest direct in-process integration and you are comfortable with native runtime setup.
- **Choose IKVM** when you want a more managed and .NET-friendly Java bridge without writing low-level native code.
- **Choose gRPC** when you want a stable, scalable, language-neutral service boundary that is easier to deploy and maintain.

Run It Using the Script

If the entire **excel** folder has been downloaded locally, open PowerShell in that folder and run the script below:

```
cd <Your project folder>\excel .\runAll.ps1
```

The script will do the following in order:

- Build the shared Java API
- Run the JNI validation test
- Run the IKVM validation test
- Start the Java gRPC service
- Run the .NET gRPC client against it
- Stop the Java service and confirm it is no longer running

Note: Make sure Java is installed and **JAVA_HOME** is set correctly before running the JNI-based example.